

DISTRIBUTED MACHINE LEARNING

Assignment 2

Implementation and Performance Analysis of the Federated Averaging (FedAvg)
Algorithm for Image Classification

Group 55

S. No.	Name	ID Number
1	Adurti Sai Venkatesh	2024ac05968
2	Akkireddy Pranith	2024ac05473
3	D. Mallikarjuna Reddy	2024ac05653

Course: Distributed Machine Learning

Course Code: AIMLCZG515

Topic: Federated Learning and Federated Averaging (FedAvg)

Dataset: MNIST Handwritten Digits

Table of Contents

Executive. Summary	3
1. Problem Statement and Objectives	3
1.1 Problem Statement	3
1.2 Objectives	3
1.3 Experimental Configuration	4
2. Dataset Description, EDA and Preprocessing	4
2.1 Dataset	4
2.2 Exploratory Data Analysis	4
2.3 Preprocessing	5
3. Client Data Partitioning Strategy	5
3.1 IID Partitioning	5
3.2 Non-IID Partitioning	5
4. Local Model Architecture	6
5. Federated Averaging Workflow	6
6. Core Implementation Logic	7
7. Execution Results: Primary IID Experiment	7
8. Experiment 1: Impact of Local Epochs	8
9. Experiment 2: IID versus Non-IID	10
10. Client Drift and Class-Wise Evaluation	11
10.1 Client Drift	11
10.2 IID Confusion Matrix	12
10.3 Non-IID Confusion Matrix	13
10.4 Class-Wise F1 Analysis	13
11. Analysis and Discussion	14
11.1 Why FedAvg Performs Differently on IID and Non-IID Data	14
11.2 Impact of Increasing Local Epochs	14
11.3 Improvement over Standard FedAvg	14
11.4 Practical Interpretation	14
12. Consolidated Results	14
13. Limitations and Practical Considerations	15
14. Conclusion	15
15. References	15

Executive. Summary

This report presents the design, implementation and evaluation of a **Federated Learning** system using **Federated Averaging (FedAvg)** for MNIST image classification. Five clients train a shared convolutional neural network without exchanging their local training images. The study evaluates both **IID** and **Non-IID** client distributions and compares local training at **1, 2 and 5 epochs** over **10 communication rounds**.

The central objective is to understand how client-side data distribution and local computation affect global model convergence. Under the primary IID configuration with two local epochs, the global model reaches **99.28% test accuracy** with **0.0254 loss**. Under the equivalent Non-IID configuration, final accuracy falls to **78.76%**, creating a **20.52 percentage-point gap**. This demonstrates that strong local fitting does not guarantee strong global generalization when clients optimize on highly skewed label distributions.

FINAL IID ACCURACY	FINAL NON-IID ACCURACY	IID VS NON-IID GAP	BEST OBSERVED IID
99.28%	78.76%	20.52 pp	99.30%

The report therefore focuses on three questions: **how FedAvg behaves under IID data**, **how increasing local epochs changes convergence**, and **why statistical heterogeneity weakens the global model**. FedProx is discussed as a natural extension because it constrains local models from drifting too far from the current global model.

1. Problem Statement and Objectives

1.1 Problem Statement

A consortium of hospitals wishes to train a common image-classification model while retaining patient data within each hospital. Centralizing the raw data is not permitted because of privacy and governance constraints. Federated learning addresses this requirement by moving the **model to the data** rather than moving the data to the model.

In each communication round, every hospital receives the current global model, trains it locally on its own records, and returns only the updated model parameters. A central coordinator combines those updates using **Federated Averaging** to create the next global model. The assignment reproduces this workflow with the **MNIST handwritten-digit dataset** and **five simulated clients**.

1.2 Objectives

- Prepare and validate the MNIST dataset, then construct both IID and Non-IID client partitions.
- Develop a compact CNN for local image classification using the required batch size and optimizer.
- Implement sample-count-weighted FedAvg manually, without any federated-learning framework.
- Run 10 communication rounds and record local-client and global-model performance.
- Compare local epochs of 1, 2 and 5 under identical IID conditions.
- Compare IID and Non-IID training and explain the effect of data heterogeneity on convergence.
- Identify an improvement over standard FedAvg for heterogeneous client data.

1.3 Experimental Configuration

Component	Configuration
Simulated clients	5
Communication rounds	10
Batch size	32
Primary local epochs	2
Local-epoch comparison	1, 2, 5
Model	Lightweight CNN
Optimizer	Adam
Loss function	Sparse categorical cross-entropy
Aggregation	Manual sample-count-weighted FedAvg
Evaluation set	MNIST test set (10,000 images)
Random seed	42

2. Dataset Description, EDA and Preprocessing

2.1 Dataset

MNIST contains **60,000 training images** and **10,000 testing images**. Each image is a **28 x 28 grayscale** representation of one digit from 0 to 9. The dataset was loaded through the TensorFlow/Keras MNIST loader and then validated before any client partitioning.

The validation checks confirmed that the arrays have the expected shapes and data types, all labels lie in the valid 0-9 range, and there are no missing values, infinite values, blank images or out-of-range pixel intensities. All **12 integrity checks passed**, so the same clean dataset is used for every experiment.

2.2 Exploratory Data Analysis

The training set is approximately balanced. Digit 1 is the most frequent class with **6,742 samples**, while digit 5 is the least frequent with **5,421 samples**; the resulting imbalance ratio is only **1.2437**. Because this imbalance is mild, no class weighting or resampling is applied.

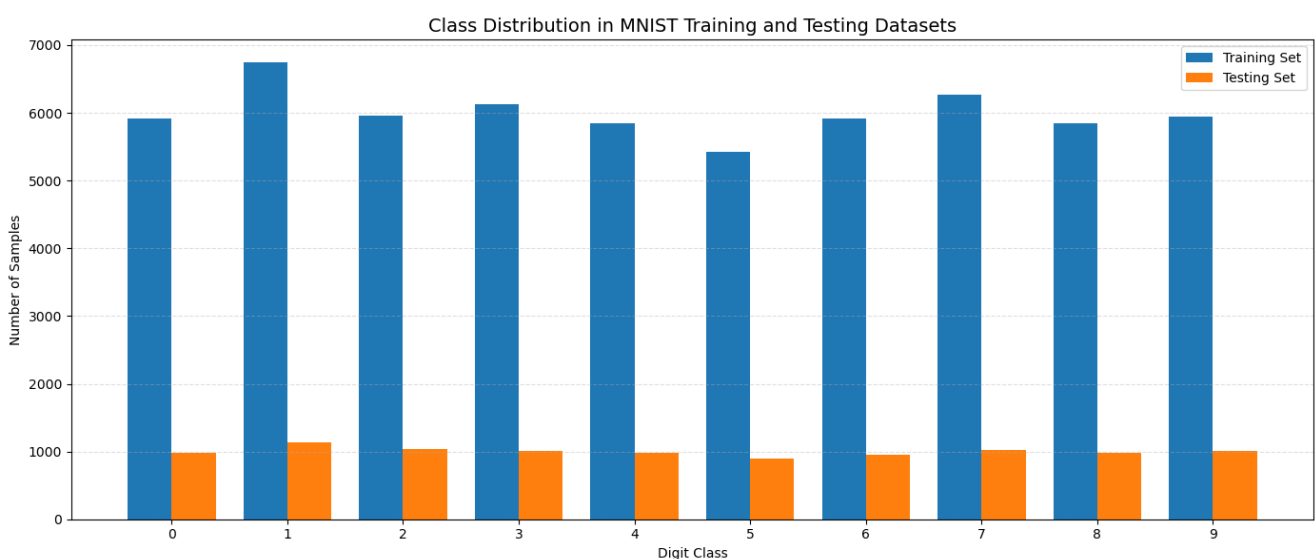


Figure 2.1: Class distribution in the MNIST training and testing sets.

Pixel-level analysis also confirms the expected sparsity of handwritten digits: most pixels belong to the dark background, while a much smaller subset represents the digit strokes. This supports the use of convolutional layers, which can exploit local spatial patterns such as edges, curves and stroke intersections.

2.3 Preprocessing

Pixel intensities are converted from the integer range 0-255 to floating-point values in **[0.0, 1.0]** by division by 255. A channel dimension is then added so each sample has shape **28 x 28 x 1**, matching the Conv2D input. The preprocessing is applied identically to all samples before the federated client splits are created.

3. Client Data Partitioning Strategy

3.1 IID Partitioning

For the IID experiment, the 60,000 training indices are shuffled using the fixed seed and divided equally among five clients. Each client therefore receives exactly **12,000 images**. Because the shuffle occurs before splitting, the class proportions remain very similar across clients.

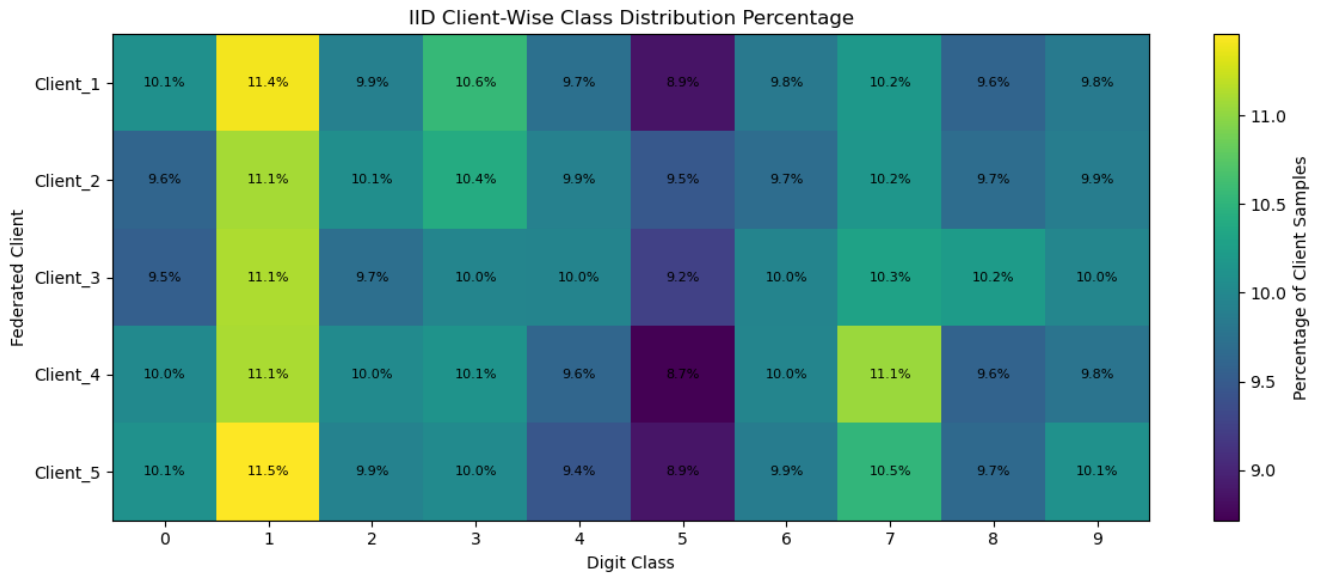


Figure 3.1: IID client-wise class percentages. Every client contains a broadly similar mixture of all ten digits.

The heatmap shows that each client's class shares remain close to the global MNIST distribution. This is important because FedAvg works best when local objectives are similar: each client is learning approximately the same classification problem, so their model updates tend to point in compatible directions.

3.2 Non-IID Partitioning

For the Non-IID experiment, labels are sorted and the data are divided into **10 shards**. Each client receives **2 shards**, producing a deliberately heterogeneous distribution in which only a few classes dominate each client's data. The total number of samples remains exactly **12,000 per client**, so the comparison isolates label distribution rather than dataset size.



Figure 3.2: Non-IID client-wise class percentages. Each client is dominated by a narrow subset of digit classes.

This partition simulates the central difficulty of practical federated learning: different institutions may observe different populations. A local model can therefore become highly specialized to its own client distribution, creating conflicting updates when the server later averages all client models.

4. Local Model Architecture

A lightweight convolutional neural network is used for both local and global models. The network accepts **28 x 28 x 1** images, applies two convolution-and-pooling stages, then uses a dense hidden layer and a softmax output layer for the ten digit classes. The complete network contains **121,930 trainable parameters**.

Layer	Configuration	Parameters
Input	28 x 28 x 1	-
Conv2D	32 filters, 3 x 3, ReLU	320
MaxPooling2D	2 x 2	0
Conv2D	64 filters, 3 x 3, ReLU	18,496
MaxPooling2D	2 x 2	0
Flatten	-	0
Dense	64 units, ReLU	102,464
Output	10 units, Softmax	650
Total		121,930

A CNN is appropriate because MNIST classes are defined by spatial stroke patterns. Convolutional filters learn local visual features, while pooling reduces spatial dimensions and improves computational efficiency. The model is deliberately compact because a fresh local instance is trained for every client in every communication round.

5. Federated Averaging Workflow

The implementation follows synchronous full-participation FedAvg. At the beginning of a round, all clients receive identical global weights. After local training, the server aggregates the returned weights layer by layer using each client's sample count.

Step	Operation	Purpose
1	Initialize / retain global model	Defines the common starting point for the round
2	Distribute global weights	Ensures every client starts from the same model state
3	Local training	Each client optimizes on its own private data
4	Collect updated weights	Only model parameters and metrics are returned
5	Weighted aggregation	Compute sample-count-weighted layer averages
6	Update global model	Replace server weights with the aggregated result
7	Global evaluation	Measure test loss and accuracy after the round

For client k with n_k samples and local weights w_k , the server computes the weighted average $\text{sum}((n_k / N) * w_k)$, where N is the total number of client samples. In this assignment every client has the same sample count, so each contributes 20%, but the implementation retains the general weighted form.

The procedure is repeated for **10 communication rounds**. After every round, the global model is evaluated on the common MNIST test set, producing one global accuracy and one global loss value that are directly comparable across experiments.

6. Core Implementation Logic

The report includes only the code fragments that establish the implementation method. These snippets show how client data are formed, how a local model is initialized from global weights, how FedAvg is performed manually, and how the server repeats the federated round.

6.1 Local client training

```
local_model = create_cnn_model()
local_model.set_weights(global_weights)

history = local_model.fit(
    client_data["x"], client_data["y"],
    epochs=local_epochs, batch_size=32,
    shuffle=True, verbose=0
)

updated_weights = local_model.get_weights()
```

Each local model starts from the current global parameters and trains only on that client's local dataset.

6.2 Manual weighted FedAvg

```
total_samples = np.sum(client_sample_counts)
avg = [np.zeros_like(w, dtype=np.float32)
       for w in client_weights[0]]

for weights, n in zip(client_weights, client_sample_counts):
    for layer, w in enumerate(weights):
        avg[layer] += (n / total_samples) * w
```

The aggregation is explicitly implemented layer by layer using the FedAvg sample-count weighting rule.

6.3 Communication-round orchestration

```
for round_number in range(1, 11):
    current = global_model.get_weights()
    local_weights, counts = train_all_clients(current)
    averaged = federated_average(local_weights, counts)
    global_model.set_weights(averaged)
    metrics = evaluate_global_model(global_model, x_test, y_test)
```

Each round follows the same distribute -> local train -> aggregate -> evaluate sequence.

7. Execution Results: Primary IID Experiment

The primary experiment uses IID clients, **2 local epochs**, batch size 32 and 10 communication rounds. The global model begins with **97.59% accuracy** after the first round and reaches **99.28%** after Round 10. The final global loss is **0.0254**, which is also the lowest loss observed in this run.

ROUND 1 ACCURACY	FINAL ACCURACY	FINAL LOSS	ROUNDS
97.59%	99.28%	0.0254	10

Round	Global Loss	Global Accuracy
1	0.0840	97.59%
2	0.0416	98.58%
3	0.0334	98.82%
4	0.0296	99.02%
5	0.0289	99.10%
6	0.0273	99.12%
7	0.0274	99.10%

Round	Global Loss	Global Accuracy
8	0.0288	99.14%
9	0.0307	99.09%
10	0.0254	99.28%

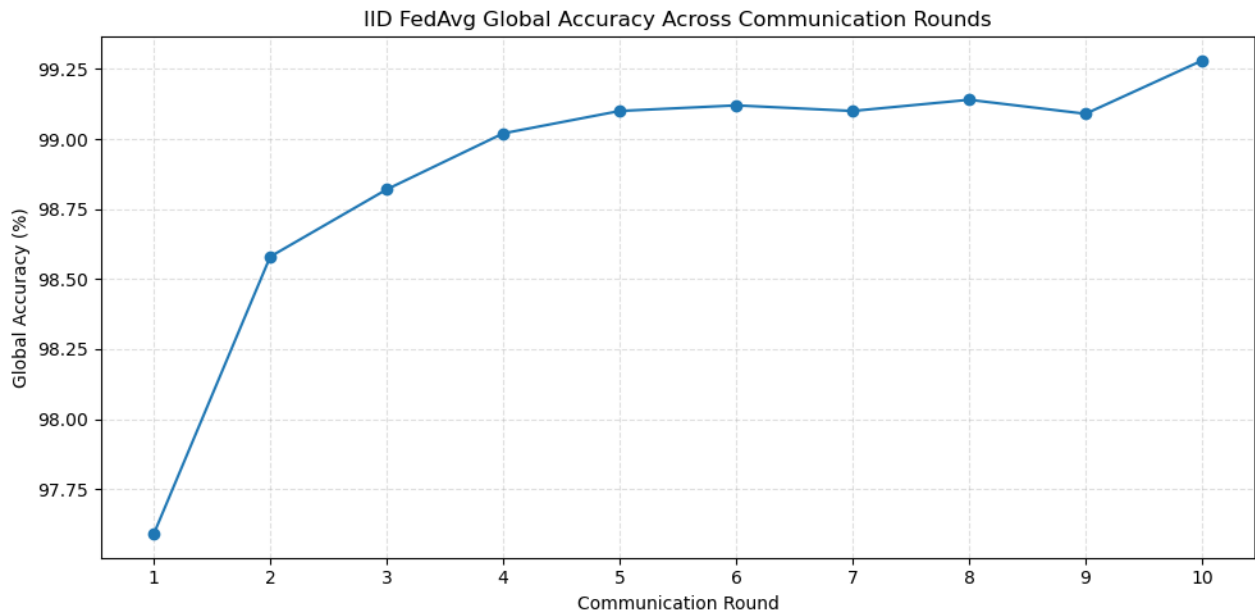


Figure 7.1: Global accuracy across the ten IID communication rounds.

Most of the accuracy gain occurs in the first few rounds. By Round 4, the model already exceeds **99%** test accuracy, and later rounds mainly refine the decision boundary. Small late-round fluctuations are expected because each round aggregates independently optimized client models rather than following a single centralized gradient trajectory.

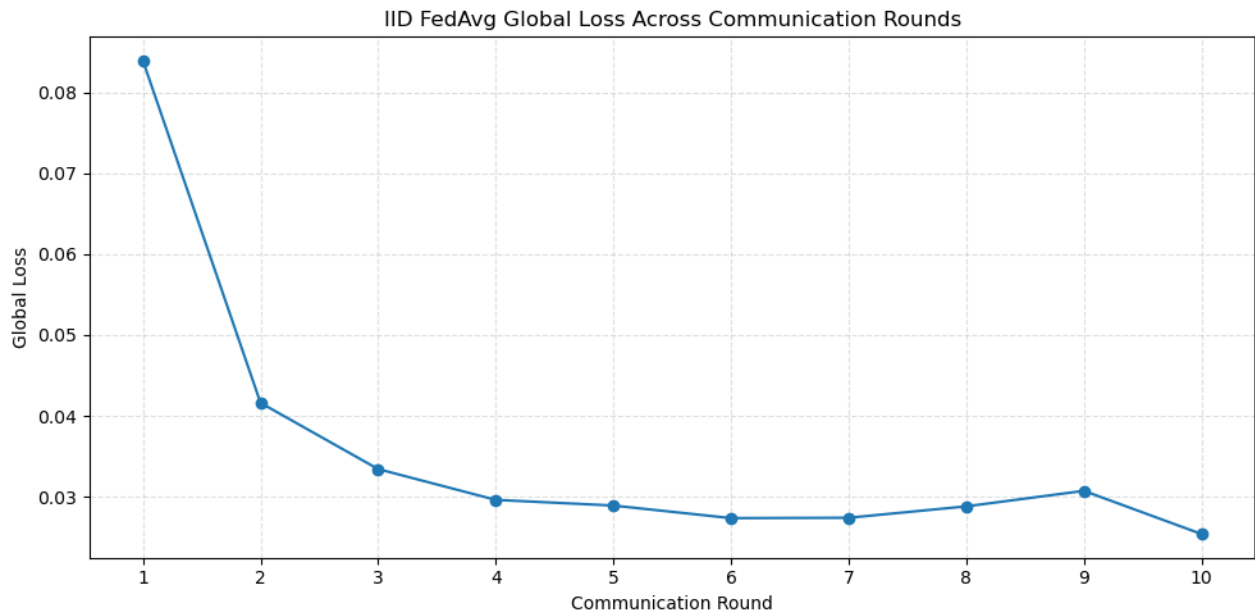


Figure 7.2: Global loss across the ten IID communication rounds.

The loss curve falls sharply from **0.0840** to **0.0416** between the first two rounds and then stabilizes at a low level. Together, the accuracy and loss trajectories indicate rapid and stable convergence under IID client data.

8. Experiment 1: Impact of Local Epochs

To isolate the effect of local computation, the IID experiment is repeated with **1, 2 and 5 local epochs** while keeping the client partitions, initial weights, batch size and communication-round count fixed.

Local Epochs	Round 1 Accuracy	Final Accuracy	Best Accuracy	Final Loss
1	96.57%	99.10%	99.17% (R9)	0.0286
2	97.59%	99.28%	99.28% (R10)	0.0254
5	98.36%	99.26%	99.30% (R9)	0.0317

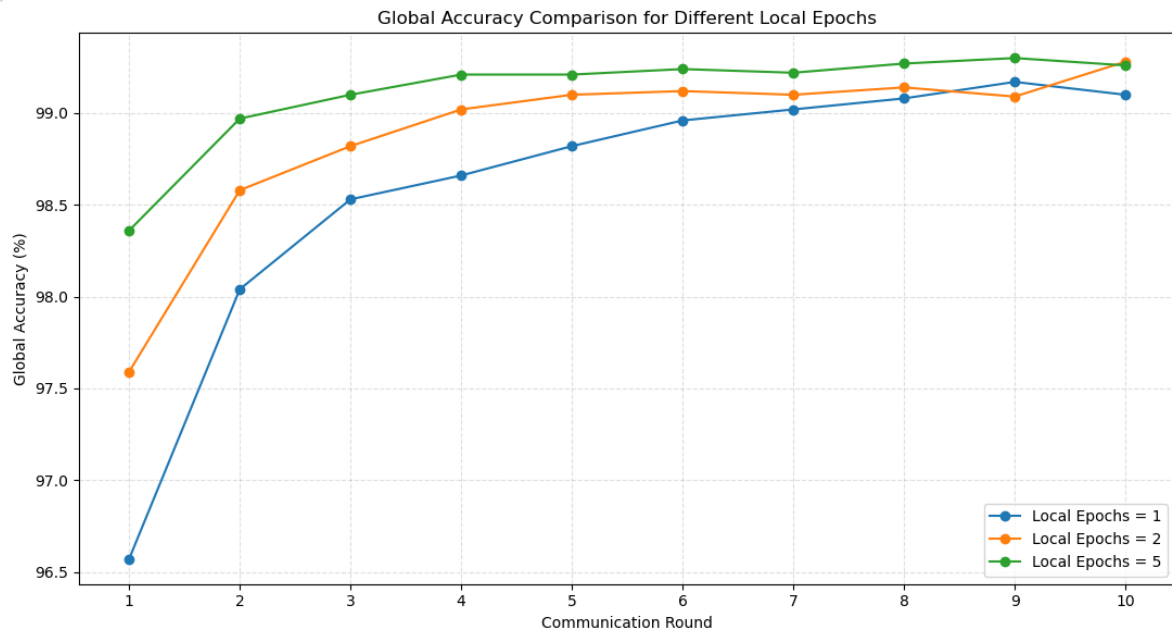


Figure 8.1: Global accuracy for 1, 2 and 5 local epochs under IID data.

Increasing local epochs improves early-round accuracy because each client performs more optimization before communicating. The 5-epoch run reaches **98.36%** in Round 1 compared with **97.59%** at 2 epochs and **96.57%** at 1 epoch.

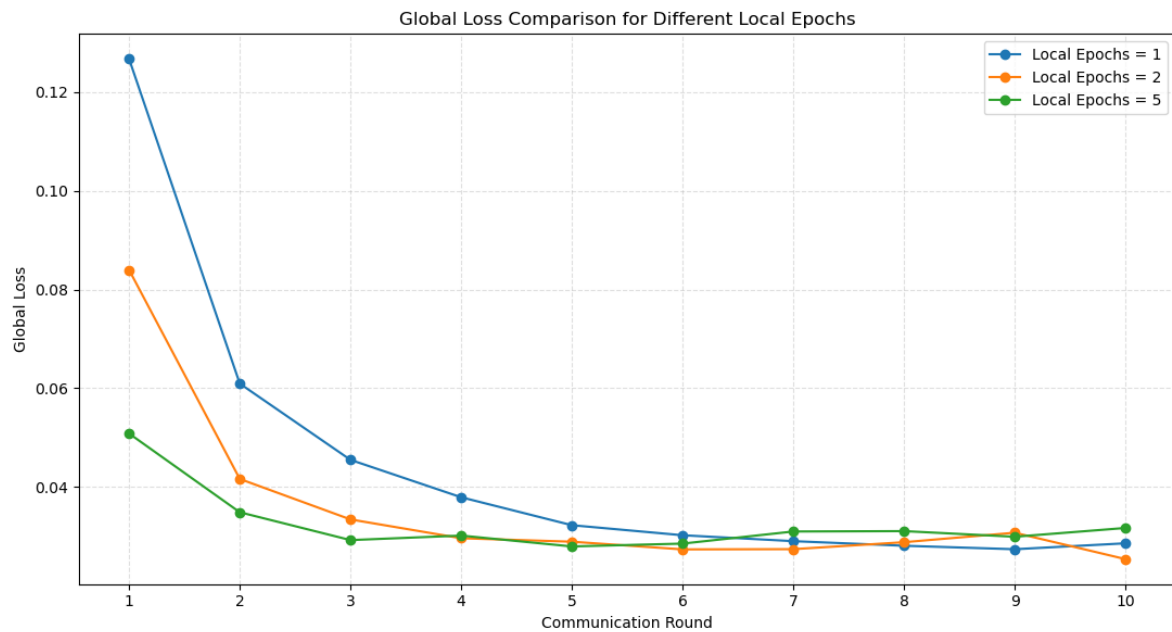


Figure 8.2: Global loss for 1, 2 and 5 local epochs under IID data.

The final accuracies, however, are very close: **99.10%**, **99.28%** and **99.26%** for 1, 2 and 5 epochs respectively. Five epochs achieve the highest single-round accuracy of **99.30%** at Round 9, but do not improve the final result. The experiment therefore shows **diminishing returns**: more local work reduces the communication needed for early progress, but beyond a point it adds computation without a meaningful final-accuracy benefit.

9. Experiment 2: IID versus Non-IID

The second experiment keeps the model architecture, reference initialization, batch size, local epochs and number of rounds identical, but replaces the IID client split with the label-shard Non-IID split. The comparison therefore isolates the effect of statistical heterogeneity.

Distribution	Round 1 Accuracy	Final Accuracy	Final Loss
IID	97.59%	99.28%	0.0254
Non-IID	41.06%	78.76%	0.6967

IID FINAL	NON-IID FINAL	ACCURACY GAP	LOSS GAP
99.28%	78.76%	20.52 pp	0.6713

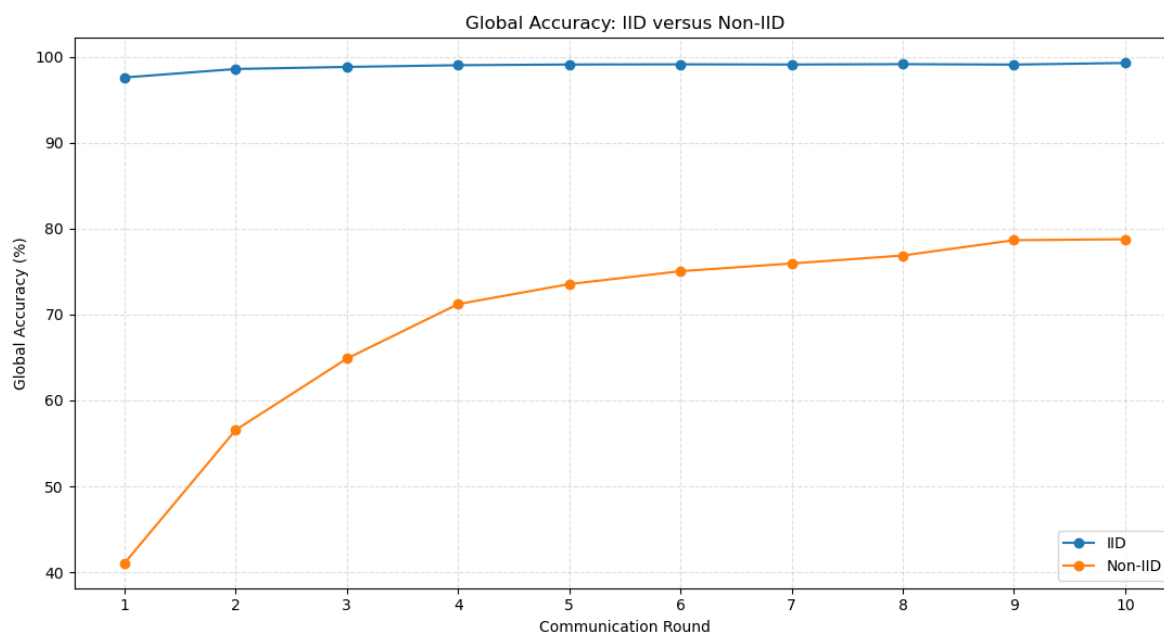


Figure 9.1: Global accuracy comparison for IID and Non-IID client distributions.

The Non-IID global model improves steadily from **41.06%** to **78.76%**, but never approaches the IID trajectory. After ten rounds, the gap remains **20.52 percentage points**. Because both experiments use the same images in total, the result is caused by how those images are distributed among clients rather than by dataset quantity.

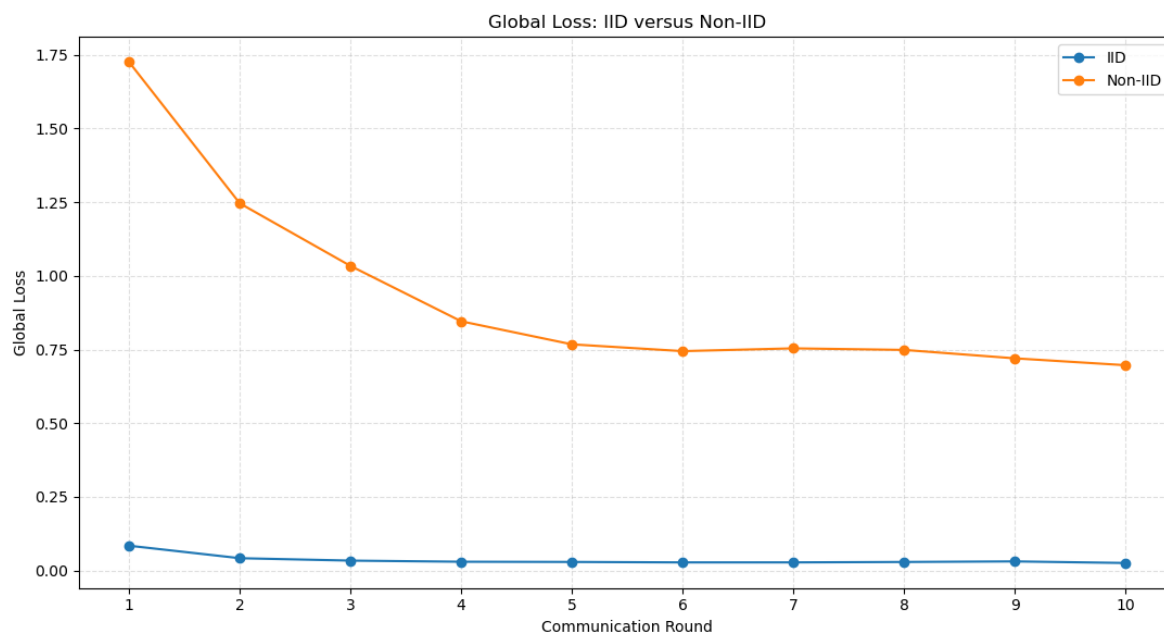


Figure 9.2: Global loss comparison for IID and Non-IID client distributions.

The loss comparison tells the same story. The IID loss falls close to zero, whereas the Non-IID model finishes at **0.6967**. Each Non-IID client optimizes heavily for its own dominant labels, so the server averages models that have learned different local objectives. The resulting global model is a compromise between specialized solutions.

10. Client Drift and Class-Wise Evaluation

10.1 Client Drift

Client drift is examined through the round-wise standard deviation of local client training accuracy. The mean dispersion is **0.081 percentage points** for IID clients and **0.193 percentage points** for Non-IID clients. Thus, Non-IID client accuracies disagree by roughly **2.39 times** more on average.

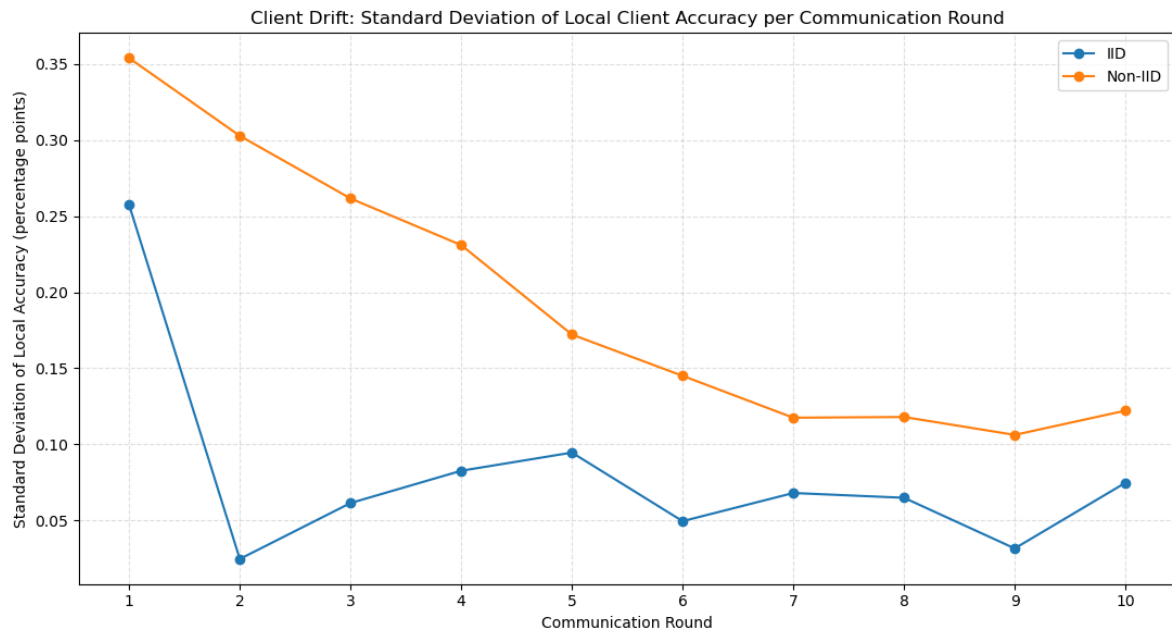


Figure 10.1: Standard deviation of local client accuracy across communication rounds.

The difference in dispersion supports the client-drift explanation. IID clients receive similar data, so local optimization moves each model in similar directions. Non-IID clients see different dominant classes, so their local solutions diverge more before aggregation.

10.2 IID Confusion Matrix

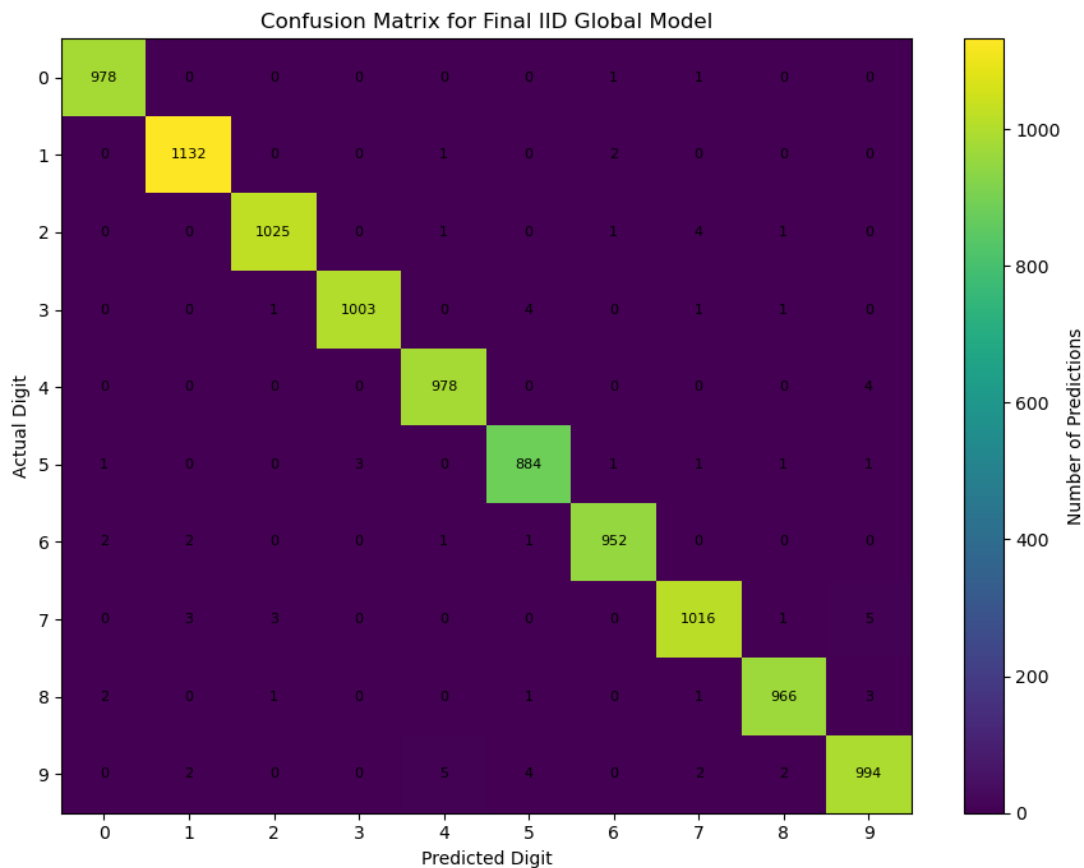


Figure 10.2: Confusion matrix of the final IID global model.

The IID confusion matrix is strongly diagonal, indicating that nearly all digit classes are recognized correctly. Per-class precision, recall and F1-score remain close to 1.0, which is consistent with the overall **99.28% accuracy**.

10.3 Non-IID Confusion Matrix

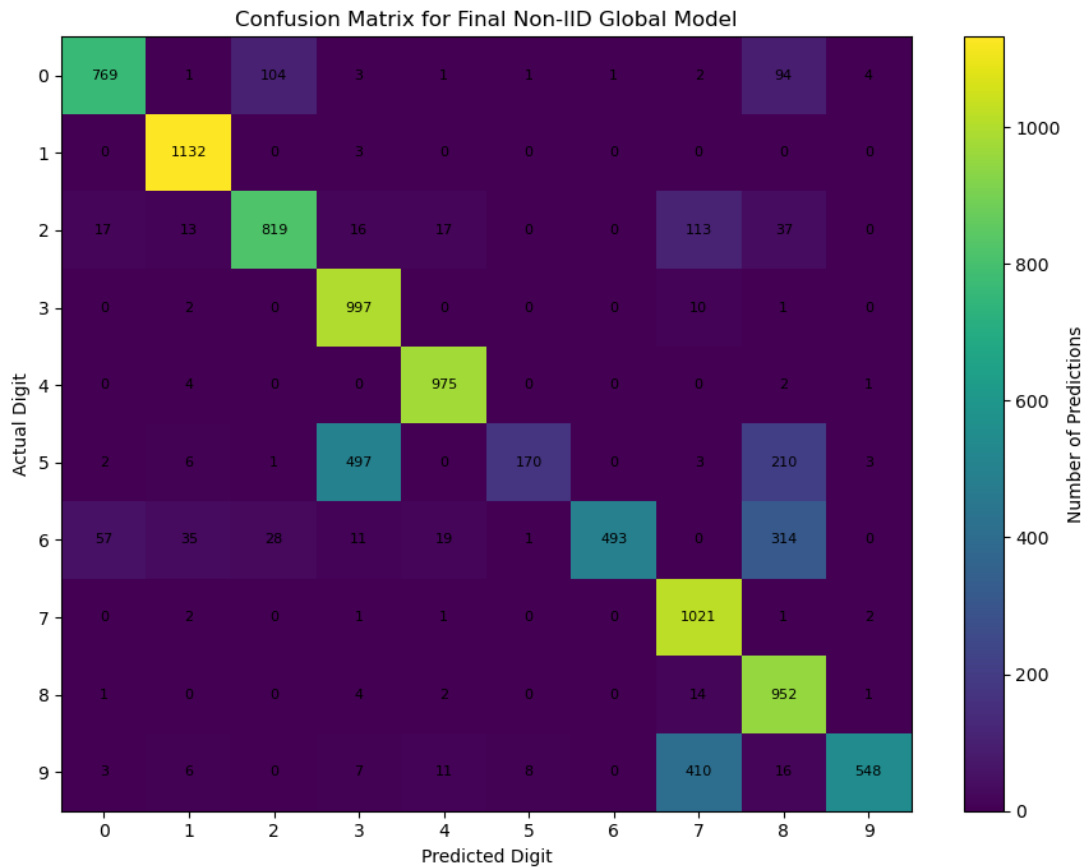


Figure 10.3: Confusion matrix of the final Non-IID global model.

The Non-IID matrix shows visibly larger off-diagonal errors. These errors are not evenly distributed across classes, which confirms that client heterogeneity creates class-specific bias rather than simply reducing all classes by the same amount.

10.4 Class-Wise F1 Analysis

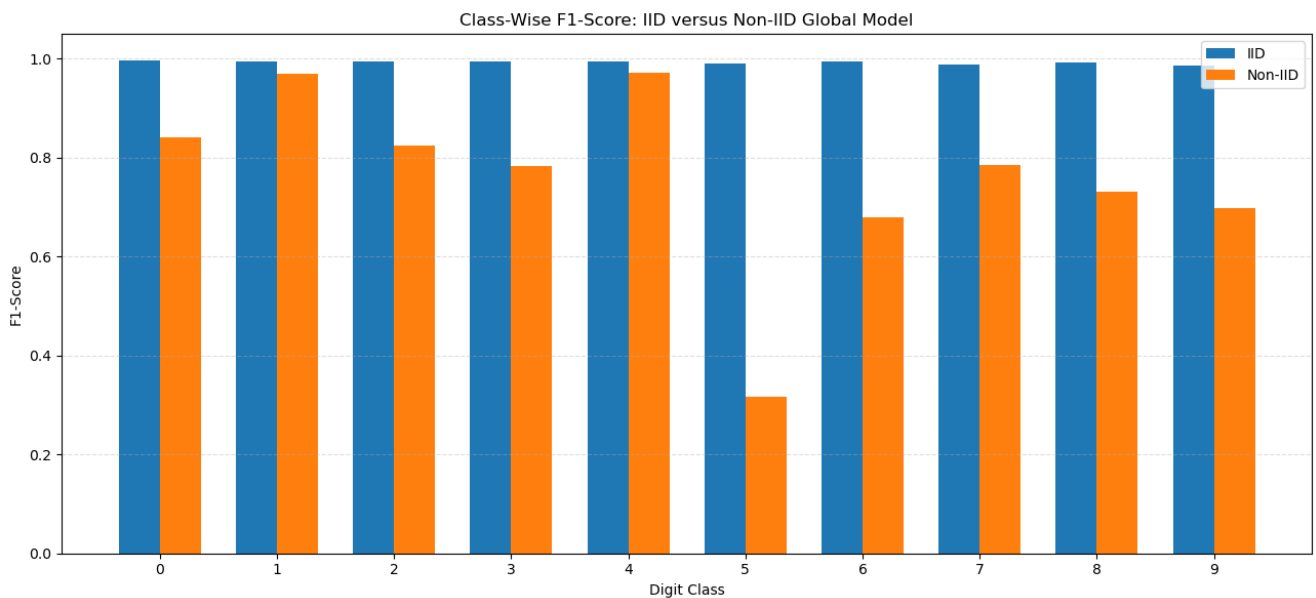


Figure 10.4: Class-wise F1-score comparison between the IID and Non-IID global models.

Digit **5** is the most affected class: its F1-score falls from **0.990** under IID training to **0.317** under Non-IID training, a reduction of **0.673**. Digits 6, 7, 8 and 9 also experience substantial degradation, while digits 1 and 4 remain comparatively strong. This uneven performance is consistent with the shard-based label concentration.

11. Analysis and Discussion

11.1 Why FedAvg Performs Differently on IID and Non-IID Data

FedAvg assumes that averaging independently trained local models produces a useful approximation to a centrally trained model. This assumption works well when the clients have similar data distributions. In the IID experiment, every client contains all ten classes in comparable proportions, so each local loss function resembles the overall global loss function. Their updates are therefore broadly aligned, and aggregation produces a stable improvement.

Under Non-IID data, the local objectives are different. A client dominated by digits 5 and 6 optimizes a different decision boundary from a client dominated by digits 1 and 8. Local training can therefore achieve almost perfect accuracy on a client's own subset while pushing the model away from features needed by other clients. Averaging these conflicting models introduces a compromise that is weaker on the complete test distribution. This is why local training accuracy can remain near 100% while global accuracy stops at **78.76%**.

11.2 Impact of Increasing Local Epochs

More local epochs reduce the frequency of communication because each client performs more learning before sending an update. That can be useful when network communication is expensive. In this IID study, higher local epochs clearly accelerate early convergence, but the final accuracy benefit is negligible. The 2-epoch configuration ends at **99.28%**, while 5 epochs end at **99.26%** despite substantially more client-side computation.

The trade-off becomes more important under heterogeneous data. When a client trains for longer on a skewed local distribution, it has more opportunity to specialize to that distribution before aggregation. Therefore, increasing local epochs can intensify client drift in Non-IID settings even though it improves local optimization.

11.3 Improvement over Standard FedAvg

FedProx addresses this limitation by adding a proximal term to the local objective. The proximal term penalizes large deviations from the current global model, so each client is discouraged from moving too far toward a highly specialized local solution. For the label-skewed setting used here, this directly targets the mechanism responsible for the IID-to-Non-IID gap.

11.4 Practical Interpretation

The experiment illustrates why federated learning performance cannot be judged from client training accuracy alone. A production system must examine global validation metrics, client heterogeneity, update divergence and class-level behaviour. A seemingly successful set of local models can still produce a biased global model if the participating clients represent very different populations.

12. Consolidated Results

Experiment	Distribution	Local Epochs	Round 1 Accuracy	Final Accuracy	Best Accuracy	Final Loss
IID - Epoch 1	IID	1	96.57%	99.10%	99.17%	0.0286
IID - Epoch 2	IID	2	97.59%	99.28%	99.28%	0.0254
IID - Epoch 5	IID	5	98.36%	99.26%	99.30%	0.0317
Non-IID - Epoch 2	Non-IID	2	41.06%	78.76%	78.76%	0.6967

BEST FINAL IID

99.28%

BEST OBSERVED IID

99.30%

FINAL NON-IID

78.76%

LARGEST F1 DROP

0.673

Across all experiments, the strongest final performance is obtained by the IID 2-epoch configuration. The 5-epoch run converges faster early and reaches a slightly higher intermediate peak, but not a stronger final result. The Non-IID run demonstrates the dominant effect of client heterogeneity: even with the same data volume and architecture, final accuracy drops by **20.52 percentage points**.

13. Limitations and Practical Considerations

The implementation is a controlled federated-learning simulation intended to demonstrate the FedAvg algorithm and its behaviour. The following limitations define the scope of the reported results.

- All five clients are simulated within one Kubeflow notebook environment rather than on physically separate systems.
- Every client participates in every communication round; client dropout and partial participation are not modeled.
- Communication latency, bandwidth, failures and model-transfer cost are not measured.
- All clients use the same compute capacity and contain equal sample counts.
- The Non-IID distribution is intentionally severe because it is created with label-based shards.
- Privacy mechanisms such as secure aggregation and differential privacy are outside the assignment scope.
- MNIST and the lightweight CNN provide a controlled benchmark; real medical-imaging workloads are larger and more heterogeneous.

14. Conclusion

This assignment implemented **Federated Averaging manually** for MNIST image classification across five simulated clients. The workflow includes global-model initialization, distribution of global weights, local client training, collection of updated weights, sample-count-weighted aggregation and evaluation after each of ten communication rounds.

Under IID data, FedAvg converges rapidly and reliably. The primary 2-epoch experiment reaches **99.28% final accuracy** with **0.0254 loss**. Increasing local epochs improves early convergence, but the final results show diminishing returns: five epochs achieve a peak of **99.30%** yet finish at **99.26%**.

The Non-IID experiment produces a much weaker final result of **78.76%**, leaving a **20.52 percentage-point gap** relative to the equivalent IID run. Client-dispersion and class-wise analysis show that this degradation is caused by local specialization and conflicting updates. Digit 5 is affected most strongly, with its F1-score falling by **0.673**.

The overall conclusion is that standard FedAvg is highly effective when participating clients have similar data distributions, but severe statistical heterogeneity can substantially weaken global generalization even when local models appear to train well. Methods such as **FedProx** are therefore important when federated clients represent heterogeneous populations.

15. References

1. LeCun, Y., Cortes, C., and Burges, C. J. C. *The MNIST Database of Handwritten Digits*.
2. McMahan, H. B., Moore, E., Ramage, D., Hampson, S., and Agueria y Arcas, B. "Communication-Efficient Learning of Deep Networks from Decentralized Data." AISTATS, 2017.
3. Li, T., Sahu, A. K., Talwalkar, A., and Smith, V. "Federated Optimization in Heterogeneous Networks." MLSys, 2020.
4. TensorFlow / Keras documentation - MNIST loading, model definition and optimization.
5. NumPy documentation - array manipulation and manual weighted aggregation.
6. Pandas documentation - experiment logging and tabular result analysis.
7. Matplotlib documentation - performance and distribution visualizations.
8. scikit-learn documentation - confusion matrices and classification reports.